# OpenShift Learning Guide

# OpenShift Learning Guide

**A Complete Walkthrough — From Containers to Production Apps on Red Hat OpenShift**

*Compiled for the OpenShift Developer Sandbox (trial cluster)*

---

## About this document

This guide is the complete content of an interactive learning conversation about Red Hat OpenShift. It walks through six phases of OpenShift mastery — from the Linux primitives underneath containers, to Kubernetes fundamentals, to OpenShift's developer platform, to deploying real applications with builds, registries, packaging, and configuration management.

Every command in this guide is calibrated to run on the **OpenShift Developer Sandbox** — Red Hat's free trial cluster — using the project `vivek-c-dev` as the working namespace. Substitute your own project name where needed.

---

## Table of Contents

**Phase 1 — Foundations** 1. Containers and Docker 2. Kubernetes Basics — Pods, Services, Deployments, Namespaces, YAML 3. Linux Namespaces — how containers are made

**Phase 2 — OpenShift Introduction** 4. OpenShift Architecture 5. OpenShift vs Kubernetes 6. RHEL CoreOS — the operating system OpenShift runs on

**Phase 3 — Working with the Cluster** 7. The Web Console and `oc` CLI 8. Projects vs Namespaces 9. RBAC basics — Users, Groups, ServiceAccounts, Roles, Bindings 10. Hands-on: Creating an RBAC binding in the Sandbox

**Phase 4 — Deploying Applications** 11. Two ways to deploy — Git (S2I) and pre-built image 12. Build Lifecycle — BuildConfig and ImageStream 13. Environment configs — ConfigMaps and Secrets

**Phase 5 — Core Platform Concepts** 14. ConfigMaps and Secrets — deep dive 15. Networking — Services, Routes, TLS termination modes 16. Storage — Persistent Volumes and PVCs

**Phase 6 — Production Patterns** 17. Deploying from internal and external registries 18. Application Templates and Helm charts 19. Environment variables and configuration management 20. Sandbox-friendly hands-on labs

**Appendix** - Sandbox capabilities and limits - Common gotchas - Useful one-liners

---

# Phase 1 — Foundations

## 1. Containers and Docker

A **container** is a way to package an application together with everything it needs to run — code, runtime, system libraries, environment variables — into a single, isolated unit that runs identically on a laptop, a server, or a cloud cluster.

The famous problem it solves: *"it works on my machine."* A container guarantees the runtime environment is bit-for-bit identical everywhere.

### How a container actually works

Containers are not lightweight VMs. They're regular Linux processes that the kernel has been told to lie to — the kernel uses three features to create the illusion of isolation:

- **Namespaces** — give the process its own view of PIDs, network interfaces, mounts, hostname. The container thinks it's PID 1 in its own little universe.
- **cgroups** — limit how much CPU, memory, and I/O the process can consume.
- **Union filesystems (overlayfs)** — stack read-only image layers and add a thin writable layer on top. This is why images are small and Docker pulls are fast.

### Containers vs Virtual Machines

| Aspect | Virtual Machines | Containers |
|---|---|---|
| Boot time | Minutes | Seconds |
| Image size | Gigabytes | Megabytes |
| Each unit has | Its own guest OS | Shares host kernel |
| Isolation | Strong (hypervisor) | Process-level (kernel) |
| Density per host | Dozens | Hundreds to thousands |
| Use case | Full OS isolation | Application packaging |

Containers share one kernel — that's why they're so much smaller and start in seconds.

### Docker — the tool that made containers usable

Docker didn't invent containers (LXC existed before it), but Docker made them *easy*. It gave us:

| Component | Purpose |
|---|---|
| **Dockerfile** | Recipe for building an image |
| **Image** | Immutable snapshot of an app + dependencies |
| **Registry** | Storage for images (Docker Hub, ECR, Quay) |
| **Container** | A running instance of an image |
| **Docker Engine** | The daemon that runs containers on a host |

A real Dockerfile for a Node.js app:

```
FROM node:18-alpine               # base layer (existing image)
WORKDIR /app
COPY package*.json ./             # cached unless deps change
RUN npm ci --only=production      # install deps as a layer
COPY . .                          # copy app source
EXPOSE 3000
CMD ["node", "server.js"]
```

The build, ship, run loop:

```
docker build -t myapp:v1 .              # build image from Dockerfile
docker push myregistry/myapp:v1         # push to registry
docker run -d -p 3000:3000 myapp:v1     # run a container
```

### Why this matters for OpenShift

OpenShift runs containers, not VMs. Every Pod you deploy in OpenShift is ultimately one or more containers built from images. Understanding Docker = understanding what OpenShift schedules.

---

## 2. Kubernetes Basics

Kubernetes is the orchestration platform OpenShift is built on. You need to understand five core concepts before OpenShift makes sense.

### Pods — the smallest deployable unit

Think of a Pod as a single apartment unit. It contains one or more containers (roommates) that share the same network and storage. Every Pod gets its own IP address inside the cluster.

**Real-world case study — an e-commerce checkout service:** Your checkout app needs two processes: a Node.js server and a logging sidecar. Instead of running them separately, you package them in one Pod. They share `localhost` — the logger reads the server's output directly.

Key Pod YAML:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: checkout-pod
  namespace: production
spec:
  containers:
  - name: app
    image: node:18-alpine
    ports:
    - containerPort: 3000
    volumeMounts:
    - mountPath: /data
      name: shared-logs
  - name: sidecar
    image: fluentd:latest
    volumeMounts:
    - mountPath: /data
      name: shared-logs
  volumes:
  - name: shared-logs
    emptyDir: {}
```

**Critical insight:** Pods are ephemeral — they can die and be replaced at any time. Never rely on a Pod's IP directly. That's what Services are for.

### Services — stable networking layer

A Service is a permanent phone number for a set of Pods. Even as Pods come and go (with new IPs each time), the Service IP stays fixed. It uses label selectors to find its Pods and load-balances traffic across them.

**Real-world case study — a payment gateway at Black Friday scale:** Your payment Pods are constantly restarting due to load. Without a Service, every client would need to track the new Pod IPs. With a Service, clients always call `payment-service:443` — the Service handles routing under the hood.

Three Service types you must know:

| Type | Accessible from | Use case |
|---|---|---|
| `ClusterIP` | Inside cluster only | Internal microservice calls |
| `NodePort` | Any node's IP + port | Dev/test exposure |
| `LoadBalancer` | Public internet | Production web traffic |

Service YAML:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: payment-service
spec:
  type: LoadBalancer
  selector:
    app: payment
  ports:
  - port: 443
    targetPort: 3000
```

## Deployments — self-healing, scalable management

A Deployment is a factory manager for your Pods. You declare the desired state ("I want 3 replicas of this image"), and the Deployment controller makes it happen — and keeps it that way. If a Pod crashes, the Deployment replaces it automatically.

**Real-world case study — rolling update at Swiggy / Zomato:** You push a new version of your restaurant-listing service. A Deployment does a rolling update: it brings up new Pods one by one, waits for each to pass health checks, then terminates old ones. Zero downtime.

The rolling update lifecycle proceeds in three steps:

1. **Step 1** — All replicas on v1, fully running.
2. **Step 2** — One v2 Pod comes up, passes health check, then one v1 is terminated. Repeat.
3. **Step 3** — All replicas on v2. Rollout complete with zero downtime.

If v2 fails health check at any point, the Deployment auto-rolls back to v1.

Deployment YAML:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: restaurant-service
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: restaurant
  template:
    metadata:
      labels:
        app: restaurant
    spec:
      containers:
      - name: api
        image: restaurant-api:v2
        ports:
        - containerPort: 8080
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 5
```

Essential commands:

```
kubectl rollout status deployment/restaurant-service
kubectl rollout undo deployment/restaurant-service
kubectl scale deployment/restaurant-service --replicas=10
```

## Namespaces — logical isolation inside one cluster

A Namespace is like separate floors in an office building. The building (cluster) is shared, but each floor (namespace) has

its own teams, resources, and access rules. Resources in one namespace are hidden from another by default.

**Real-world case study — a SaaS startup running dev / staging / prod on one cluster:**

| Namespace | Who uses it | Resource limits |
|---|---|---|
| `production` | Live customers | 16 CPUs, 32 GB RAM |
| `staging` | QA team | 4 CPUs, 8 GB RAM |
| `development` | Engineers | 2 CPUs, 4 GB RAM |
| `monitoring` | Ops tooling (Prometheus) | Cluster-wide visibility |

Namespace commands:

```
kubectl create namespace staging
kubectl get pods -n staging
kubectl get pods --all-namespaces

# Cross-namespace service DNS:
# http://payment-service.production.svc.cluster.local
#              ^service  ^namespace
```

### YAML basics — the language of Kubernetes

Every Kubernetes resource is declared in YAML. The structure is always the same four top-level fields:

```
apiVersion: apps/v1          # which API group handles this resource
kind: Deployment             # what type of resource
metadata:                    # identity
  name: my-app
  namespace: production
  labels:
    app: my-app
    version: "2.1"
spec:                        # desired state — varies by kind
  replicas: 3
```

The three YAML rules that trip everyone up:

1. **Indentation is structure** — use 2 spaces, never tabs. Wrong indent = wrong parent.
2. **Lists use** `-` — any time you see a `-`, it's an array element (containers, ports, volumes).
3. **Labels are matchmakers** — a Deployment's `spec.selector.matchLabels` must exactly match the Pod template's `metadata.labels`, or the Deployment won't manage the Pods.

### How it all connects — the full picture

Here's the mental model for OpenShift / Kubernetes development:

1. You write a **Deployment YAML** declaring your app image and replica count
2. The Deployment creates a **ReplicaSet**, which creates **Pods**
3. Each Pod runs inside a **Namespace** that enforces resource limits and access control
4. A **Service** gets a stable DNS name and routes traffic to all healthy Pods using label matching
5. External traffic hits a `LoadBalancer` Service (or Ingress), which routes to the Service, which routes to Pods

**The golden rule:** Pods are cattle, not pets. They die, they restart, they scale. Design everything around this — use Services for addressing, Deployments for lifecycle, and Namespaces for isolation.

---

## 3. Linux Namespaces — how containers are made

A **Linux namespace** is a kernel feature that gives a process a private, isolated view of one specific kind of system resource. The process thinks it has the whole machine to itself — but really, it's just been handed a customized lens through which it sees only what it's allowed to see. The kernel does the lying for it.

This is one of the two ingredients (along with cgroups) that turn a regular Linux process into what we call a "container." Docker, containerd, CRI-O, and OpenShift all rely on it.

There are seven namespace types in modern Linux.

### PID namespace — “I am PID 1”

The PID namespace gives a process its own private process ID tree. Inside, your container’s main process sees itself as PID 1 (the init process). It cannot see any process outside its namespace, because they don’t exist as far as it’s concerned.

```
# On the host machine
$ ps -ef | wc -l
247                                  # 247 processes running on this host

# Run a container
$ docker run -it --rm alpine sh

# Inside the container
/ # ps -ef
PID   USER     TIME  COMMAND
    1 root      0:00 sh              # the shell IS pid 1
    7 root      0:00 ps -ef
/ # ps -ef | wc -l
3                                    # only sees itself + headers
```

**Why this matters in OpenShift:** every Pod’s first container becomes PID 1 in the Pod’s PID namespace. This is why container apps need to handle SIGTERM gracefully — they’re the init process, and there’s no parent to clean up zombies.

### Network namespace — your own network stack

The network namespace gives each container its own network interfaces, IP addresses, routing table, and port space. Two containers can both bind to port 80 on the same host because they’re in different network namespaces.

**Why this matters in OpenShift:** every Pod gets one network namespace shared by all containers in the Pod. That’s exactly why containers in the same Pod can talk over `localhost` — they share the network namespace. Containers in *different* Pods cannot reach each other on localhost; they need the Pod IP or a Service. This is the technical reason for the entire “Pod = network unit” rule in Kubernetes.

### Mount namespace — your own filesystem

The mount namespace gives a process its own view of which filesystems are mounted where. The container sees only its own image’s filesystem (rootfs) plus whatever volumes were mounted in. It cannot see the host’s `/etc`, `/home`, `/var/log`, or anything else that wasn’t explicitly mounted.

**Why this matters in OpenShift:** when you mount a ConfigMap or Secret as a volume at `/etc/config`, that mount only exists inside the Pod’s mount namespace. The host doesn’t have it.

### UTS namespace — your own hostname

The UTS namespace isolates the hostname and NIS domain name. Each Pod gets its own UTS namespace, which is why `hostname` inside a Pod returns the Pod name, not the node name.

### IPC namespace — your own shared memory

The IPC namespace isolates System V IPC objects (shared memory, semaphores, message queues). Without IPC isolation, two PostgreSQL Pods on the same node would conflict over IPC keys.

### User namespace — root inside, nobody outside

This is the most security-relevant namespace. A user namespace remaps UIDs and GIDs — a process can be UID 0 (root) inside the namespace, but mapped to an unprivileged UID like 100000 outside.

**Why this matters in OpenShift:** OpenShift’s Security Context Constraints (SCC) leverage user namespaces. By default, OpenShift assigns each Project a random UID range and runs your containers as a UID from that range — *not* as root. Even if your container’s image expects to run as root, SCC overrides it. This is the single biggest security difference between OpenShift and vanilla Kubernetes.

### cgroup namespace — your own resource view

The cgroup namespace hides the host’s cgroup hierarchy and shows the container only its own slice. This is the newest namespace (Linux 4.6, 2016).

### How they all combine

When `docker run` or OpenShift starts a container, the kernel creates **all seven namespaces in one operation** (the `clone()` syscall with the namespace flags) and puts the new process inside all of them simultaneously.

The mind-bending takeaway: **a container is not a thing.** There's no "container object" in the kernel. A container is just a regular Linux process that the kernel was asked to create with seven specific namespace flags set.

---

# Phase 2 — OpenShift Introduction

## 4. OpenShift Architecture

**OpenShift** is Red Hat's enterprise Kubernetes platform. It bundles upstream Kubernetes with a stack of additional components that turn a raw cluster into a complete application platform — developer tools, CI/CD, monitoring, security, registry, and a polished web UI — all integrated and supported.

The shortest possible explanation: **OpenShift is Kubernetes + everything you'd otherwise have to assemble yourself.**

OpenShift uses the same control-plane / worker-node split as Kubernetes, but adds opinionated layers on top. From bottom to top:

### Infrastructure layer

- **RHEL CoreOS** — immutable, container-optimized OS designed by Red Hat specifically for OpenShift
- **CRI-O** — the container runtime (lighter than Docker, purpose-built for Kubernetes)
- Runs on bare metal, AWS, Azure, GCP, vSphere

### Kubernetes core (unchanged)

- API server, Scheduler, Controller manager, etcd
- Pods, Services, Deployments, Namespaces, ConfigMaps, Secrets

### Platform services (OpenShift adds these)

- **Security Context Constraints (SCC)** — no root by default, plus RBAC and OAuth
- **Built-in monitoring** — Prometheus and Grafana pre-configured
- **Logging** — Loki / EFK aggregation
- **Service mesh** — Istio-based traffic management with mTLS

### Developer services (OpenShift's signature layer)

- **S2I builds** — source code becomes container image without a Dockerfile
- **Routes** — HTTPS exposure with automatic TLS, smarter than Ingress
- **Internal image registry** — built-in Quay-based registry, integrated with auth
- **Pipelines + GitOps** — Tekton and ArgoCD bundled

### Developer + admin interfaces

- Web console with Developer and Administrator perspectives
- `oc` CLI (a superset of `kubectl`)
- REST API and IDE plugins

---

## 5. OpenShift vs Kubernetes

This is the question every developer asks. The answer: OpenShift *is* Kubernetes — plus opinions, plus enterprise polish, plus support.

| Aspect | Kubernetes | OpenShift |
|---|---|---|
| **Source** | Open-source project (CNCF) | Red Hat product built on Kubernetes |
| **Installation** | DIY — assemble networking, storage, ingress, etc. | Opinionated installer, single command |
| **CLI** | `kubectl` | `oc` (superset of `kubectl`) |
| **External access** | Ingress + manage your own controller | Routes — built-in, automatic TLS |
| **Build images** | Bring your own (Docker, Buildah, Kaniko) | S2I built in — Git URL to image |
| **Image registry** | None — bring your own | Built-in registry, integrated with auth |
| **Security default** | Permissive (containers can run as root) | Strict (no root by default, SCC enforced) |
| **CI/CD** | Bring your own | Tekton Pipelines + ArgoCD bundled |
| **Monitoring** | Bring your own | Prometheus + Grafana built in |
| **Web UI** | Basic Dashboard (optional) | Full developer + admin web console |
| **OS** | Any Linux | RHEL CoreOS (immutable, auto-updating) |
| **Support** | Community | Red Hat enterprise support, certified |
| **Cost** | Free | Subscription license |
| **Update path** | Manual cluster upgrades | Over-the-air, declarative cluster updates |

### When does OpenShift make sense?

**Real case study — a regional bank's containerization journey:** The bank standardizes on OpenShift because regulators demand audit trails and a hardened security baseline (SCC + RBAC out of the box), they need vendor support with SLAs, and developers want self-service deployments without learning to build a Kubernetes platform from scratch. They trade open-source flexibility for operational predictability — a fair exchange for a regulated industry.

**When Kubernetes makes more sense:** A startup with strong DevOps engineers, no compliance burden, and a preference for open-source tooling. They can pick their own ingress controller, registry, and CI system — and they want that freedom.

### The bottom line

If you can write a Deployment YAML for Kubernetes, you can deploy it on OpenShift without a single change. Everything you learned about Pods, Services, Deployments, and Namespaces applies directly. OpenShift just gives you the rest of the app platform — the integrated registry, the developer tools, the security defaults, the routing, the monitoring — without you having to build it.

---

## 6. RHEL CoreOS — the operating system OpenShift runs on

**Red Hat Enterprise Linux CoreOS (RHCOS)** is a stripped-down, container-optimized version of RHEL that Red Hat built specifically to run OpenShift. It's the operating system installed on every node — control plane and workers alike.

The short pitch: **RHCOS is RHEL with everything you don't need for running containers ripped out, and everything you do need (kubelet, CRI-O, automatic updates) baked in.**

### The four properties that define RHCOS

**1. Immutable** — `/usr` is mounted read-only. You cannot install a package on a running RHCOS node. Configuration changes go through Ignition at boot or the Machine Config Operator at runtime, never by editing files on a live node.

**2. Transactional updates with `rpm-ostree`** — OS upgrades are atomic, like Git commits. They either fully succeed or don't happen at all. Single-command rollback to the previous OS version.

**3. Container-first, with everything else removed** — only the minimal kernel, systemd, CRI-O, kubelet, and a handful of essentials. No Docker, Python, gcc, or wget on the host. If you need any of those, run them in a container.

**4. Operator-managed** — you don't SSH into RHCOS for routine work. The Machine Config Operator (MCO) drives node config declaratively via `MachineConfig` objects.

## How RHCOS differs from regular RHEL

| Aspect | RHEL (regular) | RHCOS |
|---|---|---|
| Filesystem mutability | Fully writable | `/usr` read-only |
| Package management | `yum` / `dnf` | `rpm-ostree` (atomic) |
| Initial config | kickstart / manual | Ignition |
| Ongoing config | SSH / Ansible | MachineConfig (declarative) |
| Container runtime | Install separately | CRI-O baked in |
| Updates | `yum update` + reboot | Cluster-driven, atomic |
| Rollback | Manual, restore backup | `rpm-ostree rollback` |
| Use case | General purpose servers | OpenShift nodes only |
| Subscription | Separate RHEL sub | Included in OpenShift |
| Login model | SSH for routine ops | SSH for emergencies only |
| Configuration drift | Common | Impossible |

## What this looks like in practice — a real upgrade story

When you click "Update" on an OpenShift cluster, the Cluster Version Operator detects the new version and starts orchestrating. It updates the operators in dependency order. The Machine Config Operator computes the new desired RHCOS image for each node pool, drains and reboots each worker one by one, with automatic rollback if any node fails health checks.

You don't write any of this orchestration. You don't SSH anywhere. You click a button and forty-five minutes later your entire cluster is on a new OS, new Kubernetes, new container runtime — with not a single application Pod restarted unnecessarily. **This is only possible because RHCOS is immutable, transactional, and operator-managed.**

## When you actually interact with RHCOS

Most OpenShift developers never touch RHCOS at all — you deploy your apps to the cluster and the OS is just the substrate. Platform engineers occasionally interact with it for:

- Tuning kernel parameters (via MachineConfig)
- Adding a custom CA cert to the node trust store (via MachineConfig)
- Debugging a node-level issue (via `oc debug node/<name>`)
- Configuring chrony, kernel modules, or container runtime settings (all via MachineConfig)

**One-line summary:** RHCOS is what an operating system looks like when it stops trying to be a server you administer and starts being a substrate that the cluster manages on your behalf.

---

# Phase 3 — Working with the Cluster

## 7. The Web Console and `oc` CLI

OpenShift gives you two completely equivalent ways to do everything: a polished graphical interface (the **Web Console**) and a powerful command-line tool (the `oc` **CLI**). Both talk to the same API server. Anything you can do in one, you can do in the other.

### The Web Console — visual, exploratory, beginner-friendly

The Web Console is OpenShift's browser-based interface. You log in to a URL like `https://console-openshift-console.apps.your-cluster.example.com` and get a full graphical environment. It has two perspectives:

**Developer perspective** — task-focused around "the thing I'm building." Topology view, +Add wizard, Builds, Pipelines, Helm, Project access.

**Administrator perspective** — cluster-wide view. Nodes, machine sets, networking, storage, operators, RBAC, monitoring dashboards, alerts, OperatorHub.

The console is excellent at three things specifically:

1. **Visual debugging** — when a Pod is in CrashLoopBackOff, the console shows events, logs, environment, and resource usage on a single screen.
2. **OperatorHub** — curated catalog of operators you can install with one click.
3. **Browsing the unknown** — clicking around teaches you the API.

### The `oc` CLI — fast, scriptable, scalable

The crucial fact: `oc` **is a superset of** `kubectl`. Every `kubectl` command works as `oc`, and `oc` adds OpenShift-specific commands on top:

| OpenShift-specific `oc` command | What it does |
|---|---|
| `oc login` | Authenticate with token, username/password, or OAuth |
| `oc new-app` | Deploy from Git or image, creating all required objects |
| `oc new-project` | Create a Project with display name and description |
| `oc expose` | Create a Route from a Service in one step |
| `oc rollout` | Manage Deployment rollouts and rollbacks |
| `oc adm policy` | Grant/revoke RBAC roles |
| `oc debug` | Launch a debug copy of a Pod with elevated privileges |
| `oc rsh` | Shorthand for `exec -it -- /bin/sh` into a Pod |
| `oc tag` | Manipulate ImageStream tags |
| `oc start-build` | Trigger a BuildConfig manually |

### The everyday `oc` commands

```
# Authenticate (use the "Copy login command" from the console)
oc login --token=sha256~abc123 --server=https://api.cluster.example.com:6443
oc whoami
oc whoami --show-server

# Project navigation
oc new-project payments-dev
oc project payments-dev
oc projects

# Deploy an app — three different styles
oc new-app https://github.com/me/api.git    # S2I from Git source
oc new-app nginx:latest                      # from a container image
oc apply -f deployment.yaml                  # from declarative YAML

# Inspect what's running
oc get pods
oc get all
oc get pods -o wide
oc describe pod my-pod

# Watch and debug
oc logs -f deployment/my-app
oc exec -it my-pod -- /bin/sh
oc port-forward svc/my-app 8080:80
oc rsh my-pod

# Expose to the world
oc expose svc/my-app
oc get route my-app

# Rollouts
oc rollout status deployment/my-app
oc rollout undo deployment/my-app
```

### Console vs CLI — task by task

| Task | Console | CLI |
|---|---|---|
| Deploy app from Git, first time | Wizard | One command |
| Restart a Deployment | Several clicks | `rollout restart` |
| Debug a CrashLoopBackOff pod | One screen | Multiple commands |
| Apply 20 YAML files in CI | Impossible | `apply -f` |
| Browse OperatorHub | Visual catalog | Painful YAML |
| Tail logs from multiple pods | One pod at a time | `logs -f -l app=x` |
| Grant RBAC to a user | Project access page | `adm policy` |
| Visualize app topology | Unique view | No equivalent |
| Filter pods by label across projects | One project at a time | `-A -l selector` |
| Open a shell in a container | In-browser terminal | `oc rsh` |
| Write a runbook for incidents | No scripting | Shell scripts |

### The killer feature that ties them together

The Web Console has a button in the top-right user menu called **“Copy login command.”** Click it, copy the displayed command, paste it into your terminal — and you’re authenticated to the same cluster on the CLI with the same identity, same permissions, same project context.

The right answer to “console or CLI?” is **don’t choose**. Keep both open and use whichever fits the task.

---

## 8. Projects vs Namespaces

A Project in OpenShift is a Kubernetes Namespace with extra benefits. Every Project *is* a Namespace under the hood, but

OpenShift wraps it with three things vanilla Namespaces don't have:

1. **Human-friendly metadata** — display name, description, requester annotation
2. **Automatic default Roles and RoleBindings** — admin, edit, view bound to whoever created it
3. **Self-service creation rules** — developers can spin up Projects without bothering a cluster admin

Working with Projects from `oc`:

```
oc new-project payments-dev \
  --display-name="Payments Service - Dev" \
  --description="Sandbox for the payments team"

oc projects                      # list projects you can see
oc project payments-dev          # switch context
oc status                        # one-screen summary
oc delete project payments-dev   # tear it all down
```

**Why Projects matter in practice:** in vanilla Kubernetes, only cluster admins can create Namespaces. In OpenShift, the platform ships with a `self-provisioner` ClusterRole bound to authenticated users — so any developer can `oc new-project` to get an isolated sandbox without filing a ticket.

**Real case study — multi-team SaaS:** A 50-engineer company maps each squad to its own Project (`auth-team`, `billing-team`, `notifications-team`). Each Project gets an automatic ResourceQuota (8 CPU, 16 GB) and the squad lead gets the `admin` role. Squads ship independently, can't accidentally take down each other's services, and the platform team only steps in for cluster-wide concerns.

---

## 9. RBAC basics

RBAC (Role-Based Access Control) answers a simple question for every API call: *should this user be allowed to do this verb on this resource in this place?* The answer comes from four object types working together.

### The four objects to know

A `Role` is a list of permissions inside one Project — verbs (get, list, create, update, delete) on resources (pods, services, deployments). A `ClusterRole` is the same idea but cluster-wide, not scoped to one Project. A `RoleBinding` connects a subject (User, Group, or ServiceAccount) to a Role inside a specific Project. A `ClusterRoleBinding` does the same thing cluster-wide.

The mental rule: **Role + RoleBinding = power inside one Project. ClusterRole + ClusterRoleBinding = power across the whole cluster.**

### The three RBAC subject types

A **Subject** is the entity trying to do something. Three categories exist:

**User** — a human. Logs in via SSO, LDAP, GitHub, htpasswd. Lives cluster-wide. Identified by username string. Used for individual access.

**Group** — a set of humans treated as one unit. A bag of usernames synced from your IdP. Used for team access. When someone joins or leaves the team, you change group membership in the IdP, not RBAC YAML across dozens of Projects.

**ServiceAccount** — a non-human identity for code running in the cluster. CI pipelines, controllers, monitoring agents, operators. Lives inside one namespace. Identified as `system:serviceaccount:<namespace>:<name>`. Authenticated by a token automatically mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token` inside Pods.

### OpenShift's three default Roles

You don't need to write Roles for most cases — OpenShift ships with three built-in ClusterRoles that cover 90% of needs:

| Role | What they can do | Typical user |
|---|---|---|
| `admin` | Full control inside the project — including granting access to others | Project owner / team lead |
| `edit` | Read + write workloads (Pods, Deployments, Services) but cannot grant access | Regular developer |
| `view` | Read-only — see everything, change nothing | Auditor, on-call observer, manager |

### Granting access — the three commands you need

```
# Grant a built-in role to a user inside a project
oc adm policy add-role-to-user edit alice -n payments-dev

# Grant to a whole group
oc adm policy add-role-to-group view payments-readonly -n payments-dev

# Grant a cluster-wide role (use sparingly!)
oc adm policy add-cluster-role-to-user cluster-admin bob

# Inspect — who can do what?
oc get rolebindings -n payments-dev
oc describe rolebinding admin -n payments-dev
oc auth can-i delete pods -n payments-dev
oc auth can-i delete pods -n payments-dev --as alice
```

### ServiceAccounts — RBAC for your apps

```
oc create sa ci-deployer -n payments-dev
oc adm policy add-role-to-user edit -z ci-deployer -n payments-dev
#                                   ^ -z means "subject is a ServiceAccount"
```

### RBAC YAML — what `oc adm policy` actually writes

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: alice-edit
  namespace: payments-dev
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: edit
  apiGroup: rbac.authorization.k8s.io
```

### How the API server actually validates

A common confusion: when you run `oc auth can-i list pods --as=system:serviceaccount:vivek-c-dev:default`, the binding's name is never compared. The API server works like this:

> *Given subject S, find every RoleBinding and ClusterRoleBinding whose `subjects` field contains S. Collect all the roles those bindings reference. Build the union of all permissions. Check if verb V on resource R is in that union.*

The binding's name is for humans only. **Things are matched by content (subjects, roleRef), not by name.** This is the same design as Kubernetes labels and selectors.

### The golden rules for RBAC

- Start with built-in `admin`, `edit`, `view` Roles — only write a custom Role when none fit.
- Bind to Groups, not individual Users — change group membership in the IdP instead of editing RBAC YAML.
- Reserve `cluster-admin` for a tiny set of platform engineers.
- Use ServiceAccounts for every app-to-API interaction; never share user credentials with workloads.

- Audit regularly with `oc get rolebindings --all-namespaces`.

---

## 10. Hands-on — Creating an RBAC binding in the Sandbox

This walkthrough uses your Sandbox project `vivek-c-dev`. Both UI and CLI paths produce the same RoleBinding object.

### Path A — Project Access page (Developer perspective)

1. Open the Sandbox console. URL like `console-openshift-console.apps.sandbox-mX.xxxx.p1.openshiftapps.com`.
2. Top-left perspective dropdown — choose **Developer**.
3. Project selector — confirm `vivek-c-dev`.
4. Left nav → **Project**.
5. Click the **Project access** tab.
6. Click **Add access**.
7. Set Subject = User (or Group, or ServiceAccount), Name = the username, Role = View / Edit / Admin.
8. Click **Save**.

### Path B — RoleBindings page (Administrator perspective)

1. Switch to **Administrator** perspective.
2. Left nav → **User Management** → **RoleBindings**.
3. Filter Project to `vivek-c-dev`.
4. Click **Create binding**.
5. Pick **Namespace role binding** (RoleBinding) — not ClusterRoleBinding.
6. Fill in Name, Namespace = `vivek-c-dev`, Role name (search for `view`), Subject (User / Group / ServiceAccount).
7. Toggle **YAML view** to see what's about to be created.
8. Click **Create**.

### Path C — CLI (fastest)

```
oc adm policy add-role-to-user view -z default -n vivek-c-dev
```

The `-z` flag tells `oc` the subject is a ServiceAccount.

### Verifying — `oc auth can-i`

The single most useful RBAC command:

```
oc auth can-i list pods \
  --as=system:serviceaccount:vivek-c-dev:default -n vivek-c-dev
# yes

oc auth can-i delete pods \
  --as=system:serviceaccount:vivek-c-dev:default -n vivek-c-dev
# no
```

Anatomy of the command: - `oc auth can-i` — "would this be allowed?" hypothetical question - `list pods` — verb + resource - `--as=...` — impersonate this identity - `-n vivek-c-dev` — scope of the question

The contrast — yes for read, no for write — is the entire RBAC system working. The API server is enforcing exactly the permissions described in YAML.

### Sandbox-specific gotchas

- The Sandbox auto-creates a few RoleBindings when your project is provisioned: `<your-username>-edit` (binds you to `edit`) and `<your-username>-rbac-edit` (binds you to a namespace-scoped Role that lets you manage RBAC inside your project).
- You **cannot** create ClusterRoleBindings — `oc auth can-i create clusterrolebindings` returns `no`.
- The Form view of Create Binding is cluttered with operator-installed ClusterRoles. Use **YAML view** or the CLI for clean RBAC creation.

---

# Phase 4 — Deploying Applications

## 11. Two ways to deploy an app

The first decision is always: **do you have source code, or do you have a pre-built image?** OpenShift handles both as first-class workflows.

### Path A — Source-to-Image (S2I) from Git

S2I is OpenShift's signature feature. You point it at a Git repo, and OpenShift inspects the source, picks a matching builder image (Node.js, Python, Java, .NET, Go, Ruby, PHP), runs your build inside that builder, and produces a ready-to-run application image — **no Dockerfile required**.

```
# the simplest possible deploy
oc new-app https://github.com/sclorg/nodejs-ex.git

# pin a specific builder version and a context dir
oc new-app nodejs:18~https://github.com/me/api.git \
  --context-dir=services/payments \
  --name=payments

# expose it to the world
oc expose svc/payments
oc get route payments
```

The single `oc new-app` command creates **five objects in one shot**: an `ImageStream` for the source, an `ImageStream` for the output, a `BuildConfig` describing how to build, a `Deployment` to run the result, and a `Service` for in-cluster networking. `oc expose` adds the sixth — a `Route` for public traffic.

### Path B — deploy a pre-built container image

Sometimes you already have an image — built by your CI, published by a vendor, or pulled from Docker Hub.

```
# from a public registry
oc new-app quay.io/acme/payments:v2 --name=payments

# from a private registry — create the pull secret first
oc create secret docker-registry quay-pull \
  --docker-server=quay.io \
  --docker-username=acme-bot \
  --docker-password=$QUAY_TOKEN
oc secrets link default quay-pull --for=pull

# pure declarative — Deployment YAML
oc apply -f deployment.yaml
```

**When to use each:** - **S2I path** — rapid prototyping, internal tools, teams without strong Docker expertise, environments where you want a consistent, hardened base image policy. - **Image path** — vendor images you don't control (Postgres, Redis, Kafka), images built by an external CI like GitHub Actions, anywhere your build pipeline already lives outside the cluster.

---

## 12. Build Lifecycle — BuildConfig and ImageStream

These two objects are the heart of OpenShift's developer experience. They're what makes `oc new-app` feel magical, and what enables the "rebuild everything that depends on this base image" workflow that's almost impossible to do cleanly in plain Kubernetes.

### What is a BuildConfig?

A `BuildConfig` is the recipe for turning source (Git, a Dockerfile, or a binary upload) into an image. It defines three things: the **source** (where the input comes from), the **strategy** (how to build it — S2I, Docker, or Custom), and the **output** (where the resulting image goes — almost always an ImageStreamTag in the internal registry).

```yaml
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: payments
spec:
  source:
    type: Git
    git:
      uri: https://github.com/acme/payments.git
      ref: main
    contextDir: services/api
  strategy:
    type: Source                      # this is S2I
    sourceStrategy:
      from:
        kind: ImageStreamTag
        name: nodejs:18-ubi9
        namespace: openshift          # builder lives in the platform namespace
  output:
    to:
      kind: ImageStreamTag
      name: payments:latest
  triggers:
  - type: GitHub
    github: { secret: webhook-secret }
  - type: ImageChange
  - type: ConfigChange
```

## The three build strategies

| Strategy | Input | Output | Use when |
|---|---|---|---|
| **Source (S2I)** | Git source + builder image | App image | You have code, no Dockerfile |
| **Docker** | Git repo with a Dockerfile | App image | You already wrote a Dockerfile |
| **Custom** | A custom build image of your choice | Anything | You want full control of the build process |

## The three trigger types

`ConfigChange` — fires when the BuildConfig itself is edited.

`ImageChange` — fires when the *builder image* updates (e.g. Red Hat publishes a new `nodejs:18-ubi9` with a CVE patch). Your app image gets rebuilt automatically. This is the "patch every app in the cluster with one operation" superpower.

`GitHub` / `GitLab` / `Bitbucket` / `Generic` — fires when a `git push` triggers a webhook.

## What is an ImageStream?

An `ImageStream` is OpenShift's pointer-and-versioning layer over container images. **It doesn't store images** — those live in the registry. It stores **stable, tagged references** to images, plus a history of what each tag pointed to over time. Think of it as Git for image tags.

The crucial property: **anything that watches an ImageStream gets notified when its tag updates.** This is what makes automatic rebuilds work.

## Walking the lifecycle

A `git push` fires the GitHub webhook configured on the BuildConfig. OpenShift creates a `Build` object (a one-off run of the BuildConfig) and schedules a build pod on a worker node. That pod pulls the source, runs the S2I assemble script inside the builder image, commits the result as a new image, and pushes it to the internal registry under the output ImageStream's tag (`payments:latest`).

The output ImageStream's `latest` tag now resolves to a new SHA. The Deployment, configured with an `image.openshift.io/triggers` annotation, sees the change and starts a rolling update.

## Useful build commands

```
oc get bc                          # list BuildConfigs
oc start-build payments            # trigger a build manually
oc start-build payments --from-dir=. --follow    # build from your local dir
oc logs -f bc/payments             # stream the build log
oc get builds                      # list recent builds
oc describe build payments-7       # full details of one build run

oc get is                          # list ImageStreams
oc describe is payments            # see all tags and their history
oc tag payments:latest payments:rollback   # promote/rollback by retagging
oc import-image nodejs:18 \
  --from=registry.access.redhat.com/ubi9/nodejs-18 --confirm
```

### Real case study — base-image security patches at scale

A platform team maintains the `nodejs:18-ubi9` ImageStream in the `openshift` namespace. When Red Hat publishes a CVE patch, the team runs a single `oc import-image` to update the ImageStream tag. **Every BuildConfig in the cluster that has an `ImageChange` trigger automatically rebuilds**, and every Deployment that watches the resulting output stream rolls out new pods. One command patches a thousand running services.

In vanilla Kubernetes, you'd be writing scripts to scan, rebuild, and redeploy each app individually. ImageStreams turn that into a graph the platform walks for you.

---

## 13. Environment configs — first look

The cardinal rule of the Twelve-Factor App: **strict separation of config from code.** Your image is identical in dev, staging, and prod — only the configuration differs. OpenShift uses two Kubernetes-native objects: `ConfigMap` for non-sensitive data, `Secret` for sensitive data.

### Creating ConfigMaps and Secrets

```
# ConfigMap from literal values
oc create configmap app-config \
  --from-literal=LOG_LEVEL=info \
  --from-literal=FEATURE_FLAG=on

# ConfigMap from a file
oc create configmap nginx-conf --from-file=nginx.conf

# Secret from literals
oc create secret generic db-creds \
  --from-literal=DB_PASSWORD='s3cr3t!' \
  --from-literal=API_TOKEN='abc123'

# Secret for a TLS cert
oc create secret tls payments-tls --cert=tls.crt --key=tls.key
```

### Wiring config into a Pod

There are two ways to inject — environment variables (simpler) or mounted files (better for certs and large config files).

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payments
spec:
  template:
    spec:
      containers:
      - name: api
        image: payments:latest
        envFrom:
        - configMapRef: { name: app-config }
        env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef: { name: db-creds, key: DB_PASSWORD }
        volumeMounts:
        - name: tls
          mountPath: /etc/tls
          readOnly: true
      volumes:
      - name: tls
        secret: { secretName: payments-tls }
```

### Real case study — three projects, one image

A team runs `payments-dev`, `payments-staging`, and `payments-prod`. The image `payments:v2.4.1` is bit-for-bit identical in all three. What differs:

| Project | DB_HOST | DB_PASSWORD | LOG_LEVEL |
|---|---|---|---|
| `payments-dev` | `dev-db.svc` | `devpass` | `debug` |
| `payments-staging` | `stg-db.acme.io` | (from Vault) | `info` |
| `payments-prod` | `prod-db.acme.io` | (from Vault) | `warn` |

Promoting from staging to prod is just `oc tag payments:staging payments:prod` — same image, different ConfigMap and Secret. No rebuild, no environment-specific Dockerfiles.

---

# Phase 5 — Core Platform Concepts

## 14. ConfigMaps and Secrets — deep dive

You saw the basics in Phase 4. Here's what was held back: the operational realities of running these at scale.

### How updates propagate at runtime

The behavior depends entirely on *how* you injected them.

**Path A — env vars (envFrom or env)** — snapshot taken at pod start. Running pods see OLD value until they restart. Required: `oc rollout restart deployment/<name>`.

**Path B — mounted files (volumes)** — kubelet syncs every ~60 seconds. Running pods see NEW value automatically. App must re-read the file or watch for inotify event. No restart needed.

**Same rules apply to Secrets.**

### Secrets — what they actually protect against

A Secret is **base64-encoded, not encrypted by default**. Anyone with `get secrets` RBAC in the namespace can read the plaintext:

```
oc get secret db-creds -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
# s3cr3t!
```

Secrets give three real benefits: 1. **Separation from images** — the value never gets baked into the container image. 2. **Audit trail** — every access goes through the API server. 3. **RBAC scoping** — you can grant `get pods` without granting `get secrets`.

What Secrets *don't* give you out of the box: encryption at rest in etcd, or protection from a node-level attacker.

For real protection:

| Layer | What it adds |
|---|---|
| Enable etcd encryption-at-rest | Secrets stored encrypted on disk in etcd |
| External Secrets Operator + Vault | Source of truth is Vault; OpenShift gets ephemeral copies |
| Sealed Secrets | Secret YAML is encrypted before commit |

### The patterns that go wrong

- **Putting non-secrets in Secrets "just in case."** Now half your team can't read deployment configs. Use ConfigMaps.
- **Storing the same DB password in 12 Secrets across 12 projects.** When it rotates, you have 12 places to update. Use a single source.
- **Committing base64-"encoded" Secret YAML to Git.** That's not encryption. Use Sealed Secrets or SOPS.

---

## 15. Networking — Services, Routes, TLS

This is the most important section in this phase. OpenShift's networking model layers cleanly: **Service for inside the cluster, Route for outside, TLS at one of three termination points.**

### The full traffic path

External user hits a hostname → DNS resolves to the cluster's router IP → HAProxy router pod receives the request → matches a Route by hostname → forwards to a Service → kube-proxy iptables routes to a Pod.

### Routes — the OpenShift way to expose HTTP/S

A Route is OpenShift's purpose-built ingress object. It predates Kubernetes Ingress and is more capable: built-in TLS termination with three modes, sticky sessions out of the box, weighted traffic splitting for canary deployments, and a

single object instead of Ingress + IngressController + manually-managed cert-manager.

```
# the simplest possible Route — auto-generated hostname, no TLS
oc expose svc/payments

# explicit hostname, edge TLS using the cluster's default cert
oc create route edge payments \
  --service=payments \
  --hostname=payments.acme.io

# bring your own cert
oc create route edge payments \
  --service=payments \
  --hostname=payments.acme.io \
  --cert=tls.crt --key=tls.key --ca-cert=ca.crt
```

The Route YAML:

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: payments
spec:
  host: payments.acme.io
  to: { kind: Service, name: payments }
  port: { targetPort: 3000 }
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

## TLS — three termination modes

**Edge** — the default. The router holds the cert, decrypts at the edge, and forwards plaintext to your pod. Simplest to operate (one cert, managed by the platform) and the right choice for 90% of public web apps.

**Passthrough** — the router uses SNI to pick the backend, then forwards the raw TLS bytes. Your pod terminates TLS itself. Use this when the application requires mutual TLS (mTLS), or when you can't share the private key with the platform team.

**Reencrypt** — the strict-compliance option. The router decrypts (so it can do L7 routing, header injection, rate limiting), then re-encrypts to the pod using a separate cert. Use this for end-to-end encryption requirements like PCI-DSS or HIPAA.

## Real case study — three Routes for one app

A healthcare SaaS deploys its patient-records API with three different Routes:

| Route hostname | Termination | Why |
|---|---|---|
| `app.acme.io` | edge | Public marketing site — fast, cluster-managed certs |
| `api.acme.io` | reencrypt | Patient-data API — HIPAA requires end-to-end encryption |
| `mtls.acme.io` | passthrough | Partner integration with mTLS client certs |

All three Routes point to the same Service. Three audiences, three security postures, one application.

## Canary deployments with Route weights

Routes support traffic splitting natively — a feature plain Kubernetes Ingress lacks:

```yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: payments
spec:
  host: payments.acme.io
  to:
    kind: Service
    name: payments-stable
    weight: 90
  alternateBackends:
  - kind: Service
    name: payments-canary
    weight: 10
```

10% of traffic goes to the canary. Monitor error rates, dial up the weight, dial it back down if something breaks.

---

## 16. Storage — Persistent Volumes and PVCs

Pods are ephemeral. Databases, file uploads, cache state, and audit logs need to outlive any single Pod. That's what Persistent Volumes solve.

### The two-object model

Storage uses a **claim-and-fulfillment pattern**. The developer (Pod) writes a *claim* describing what they need ("5GB, fast SSD, read-write"). The platform (StorageClass) provisions a *volume* that matches. The Pod mounts it.

The split lets developers and platform engineers work independently. A developer writes a PVC ("I need 5GB") without knowing whether the cluster runs on AWS EBS, Azure Disk, NetApp, or Ceph.

### Writing a PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: gp3
```

Mount it in your Pod:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  template:
    spec:
      containers:
      - name: postgres
        image: postgres:16
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: postgres-data
```

### Access modes — the three letters that matter

| Mode | Acronym | Meaning | Typical backend |
|---|---|---|---|
| ReadWriteOnce | RWO | One node can read/write at a time | Block storage (EBS, Azure Disk) |
| ReadOnlyMany | ROX | Many nodes, read-only | Pre-populated reference data |
| ReadWriteMany | RWX | Many nodes, all read/write | NFS, CephFS, Azure Files |
| ReadWriteOncePod | RWOP | Exactly one Pod | Same as RWO |

The trap: **most cloud block storage is RWO only.** If you write a Deployment with `replicas: 3` and a single RWO PVC, two of your three pods will fail to start.

### Reclaim policies

| Policy | What happens to the data |
|---|---|
| `Delete` | PV is deleted, cloud volume is deleted, data is gone forever |
| `Retain` | PV stays but goes to “Released” state; data is preserved |

For dev/test, `Delete` is fine. For production databases, **always use** `Retain`.

### StatefulSet — when you need ordered, stable storage

A StatefulSet gives each replica a stable identity (`postgres-0`, `postgres-1`, `postgres-2`) and **its own PVC** — automatically created from a `volumeClaimTemplate`.

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 3
  selector: { matchLabels: { app: postgres } }
  template:
    metadata: { labels: { app: postgres } }
    spec:
      containers:
      - name: postgres
        image: postgres:16
        volumeMounts:
        - { name: data, mountPath: /var/lib/postgresql/data }
  volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: [ReadWriteOnce]
      resources: { requests: { storage: 20Gi } }
      storageClassName: gp3-retain
```

Three Pods, three PVCs, three independent volumes — exactly what a database cluster needs.

### The storage rules to internalize

- Use Deployments for stateless apps, StatefulSets for stateful workloads.
- Use `Retain` reclaim policy for production data.
- Match access mode to replica count — RWO with multi-replica is the most common storage bug.
- Pick `WaitForFirstConsumer` binding mode with multiple AZs to avoid scheduling pods where their volumes can’t attach.
- Back up your PVs with snapshots — replication isn’t backup.

---

# Phase 6 — Production Patterns

## 17. Deploying images from internal and external registries

Every container image OpenShift runs lives in some **registry** — a server that stores images by name and tag. Where the image comes from changes how you reference it, what authentication you need, and how rollouts behave.

### Internal registry

The cluster runs its own registry at `image-registry.openshift-image-registry.svc:5000`. Every image built by an S2I BuildConfig lands there. Pods pull from it automatically using their ServiceAccount token, no credentials to manage.

```
oc get is hello-s2i -o jsonpath='{.status.dockerImageRepository}'
# image-registry.openshift-image-registry.svc:5000/vivek-c-dev/hello-s2i
```

### Public external — easiest deploy you can do

```
# Pull image directly — no build, no Dockerfile, no login
oc new-app --name=public-nginx \
  --image=nginxinc/nginx-unprivileged:latest

# Expose with TLS
oc create route edge public-nginx --service=public-nginx --port=8080-tcp
```

Notice we used `nginxinc/nginx-unprivileged`, not the regular `nginx`. The regular Nginx image expects to run as root and binds port 80, both of which violate Sandbox SCC. The unprivileged variant runs as UID 101 and listens on 8080 — it works on OpenShift out of the box.

**This is the #1 reason public images fail on the Sandbox.** Always look for `unprivileged`, `non-root`, or UBI-based variants.

### Private external — the credentials dance

Private registries need a **pull secret**:

```
# Create a pull secret for a private registry
oc create secret docker-registry quay-pull \
  --docker-server=quay.io \
  --docker-username=acme-bot \
  --docker-password='your-token-here' \
  --docker-email=ci@acme.io

# Tell the default ServiceAccount to use it for image pulls
oc secrets link default quay-pull --for=pull

# Now Pods in this project can pull from quay.io/acme/private-image
oc new-app --name=private-app --image=quay.io/acme/private-image:v1
```

The `oc secrets link default quay-pull --for=pull` step is the one people forget — without it, the secret exists but no Pod knows to use it.

### Hands-on Lab 1 — Three deploys, three sources

Run all three on your Sandbox. Each shows a different registry path:

```
oc project vivek-c-dev

# A. From the INTERNAL registry — reuse the image you already built
oc new-app --name=internal-redeploy \
  --image-stream=hello-s2i:latest
oc create route edge internal-redeploy \
  --service=internal-redeploy --port=8080-tcp

# B. From a PUBLIC external registry
oc new-app --name=public-nginx \
  --image=nginxinc/nginx-unprivileged:latest
oc create route edge public-nginx \
  --service=public-nginx --port=8080-tcp

# C. From a public Red Hat registry
oc new-app --name=ubi-httpd \
  --image=registry.access.redhat.com/ubi9/httpd-24:latest
oc create route edge ubi-httpd \
  --service=ubi-httpd --port=8080-tcp

# See all three running
oc get pods,routes
```

**Cleanup before the next section:**

```
oc delete all -l app=internal-redeploy
oc delete all -l app=public-nginx
oc delete all -l app=ubi-httpd
```

---

# 18. Application Templates and Helm charts

Real applications are bundles of 5, 10, 20 related objects, and you'll deploy variations of them in dev, staging, prod, multiple regions, multiple customers. Copy-pasting YAML is unsustainable.

OpenShift gives you two tools for this packaging problem: **Templates** (the older, OpenShift-native answer) and **Helm charts** (the newer, Kubernetes-ecosystem standard).

## OpenShift Templates

A Template is OpenShift's native packaging format — a single YAML file containing a list of objects with placeholders, plus a `parameters` section.

```yaml
apiVersion: template.openshift.io/v1
kind: Template
metadata:
  name: hello-app
parameters:
- name: APP_NAME
  value: "hello"
  required: true
- name: HOSTNAME
  required: true
- name: REPLICAS
  value: "1"
- name: DB_PASSWORD
  generate: expression
  from: "[a-zA-Z0-9]{16}"
objects:
- apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: ${APP_NAME}
  spec:
    replicas: ${{REPLICAS}}
    selector: { matchLabels: { app: ${APP_NAME} } }
    template:
      metadata: { labels: { app: ${APP_NAME} } }
      spec:
        containers:
        - name: app
          image: nodejs-ex:latest
          env:
          - name: DB_PASSWORD
            value: ${DB_PASSWORD}
- apiVersion: v1
  kind: Service
  metadata:
    name: ${APP_NAME}
  spec:
    selector: { app: ${APP_NAME} }
    ports: [{ port: 8080, targetPort: 8080 }]
```

Using a Template:

```bash
# Upload the template definition once
oc create -f hello-app-template.yaml -n vivek-c-dev

# Instantiate it
oc new-app hello-app -p APP_NAME=hello-dev -p HOSTNAME=hello-dev.apps...

# Or process and apply (single shot, no upload)
oc process -f hello-app-template.yaml \
  -p APP_NAME=hello-stage \
  -p HOSTNAME=hello-stage.apps... | oc apply -f -

# See the cluster-wide templates Red Hat ships
oc get templates -n openshift
```

## Helm charts

Helm is the package manager for Kubernetes — think `apt` for clusters. A Helm chart is a folder with templated YAMLs and a `values.yaml`.

Chart structure:

```
hello-chart/
├── Chart.yaml              # metadata: name, version, description
├── values.yaml             # default parameter values
├── templates/              # YAML templates (Go template syntax)
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── route.yaml
│   ├── configmap.yaml
│   └── _helpers.tpl
└── charts/                 # nested chart dependencies
```

`Chart.yaml` is the manifest:

```
apiVersion: v2
name: hello-chart
description: A Node.js app on OpenShift
version: 1.0.0
appVersion: "2.4.1"
```

`templates/deployment.yaml` uses Go templating:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: app
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        env:
        - name: GREETING
          value: {{ .Values.config.greeting | quote }}
        {{- if .Values.config.logLevel }}
        - name: LOG_LEVEL
          value: {{ .Values.config.logLevel | quote }}
        {{- end }}
```

Three things you can't do with Templates:

1. **Conditionals** — `{{- if ... }}` only renders fields when conditions are met.
2. **Pipelines and functions** — `| quote`, `| default`, `| toYaml`, `| b64enc`.
3. **Built-in context** — `.Release.Name`, `.Release.Namespace`, `.Chart.Version`.

## Templates vs Helm — when to pick which

| Aspect | Templates | Helm charts |
|---|---|---|
| Native to | OpenShift only | All Kubernetes |
| Templating power | Simple substitution | Full Go templating |
| Conditionals / loops | No | Yes |
| Upgrade lifecycle | No | `helm upgrade` |
| Rollback | No | `helm rollback` |
| Public ecosystem | Red Hat-curated only | Huge (Artifact Hub) |
| Portable to other K8s | No (uses Routes) | Yes |
| Console integration | +Add → All services | +Add → Helm Chart |
| Learning curve | Low | Medium |
| Best for | Simple OpenShift apps | Complex stacks · upgrades |
| Status in 2026 | Legacy, still supported | Industry standard |

## Hands-on Lab 2A — Use a Red Hat-provided Template

```
# See what's available
oc get templates -n openshift | head -20

# Pick a simple one — describe it first to see parameters
oc describe template nodejs -n openshift | head -40

# Process it with your parameter values, pipe to apply
oc process -n openshift nodejs \
  -p NAME=template-demo \
  -p SOURCE_REPOSITORY_URL=https://github.com/sclorg/nodejs-ex.git \
  | oc apply -f -

# Watch it deploy
oc logs -f bc/template-demo
oc get pods -l app=template-demo
```

Cleanup:

```
oc delete all -l app=template-demo
oc delete is template-demo
```

### Hands-on Lab 2B — Install a Helm chart

```
# Add the Bitnami repo
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Search
helm search repo bitnami | head -10

# Install Apache web server
helm install demo-apache bitnami/apache \
  --set service.type=ClusterIP \
  --set replicaCount=1 \
  --set persistence.enabled=false \
  --set image.security.allowInsecureImages=true

# Watch it come up
oc get pods -l app.kubernetes.io/instance=demo-apache -w

# See what Helm tracks
helm list

# Upgrade — change a value
helm upgrade demo-apache bitnami/apache \
  --reuse-values --set replicaCount=2

# Helm now tracks two revisions
helm history demo-apache

# Rollback to revision 1
helm rollback demo-apache 1

# Uninstall — removes everything Helm created
helm uninstall demo-apache
```

The lifecycle is the point: install, upgrade, rollback, uninstall — all by name, all tracked. **No equivalent exists for Templates.**

---

## 19. Environment variables and configuration management — full mastery

You've already touched `oc set env`. Now let's do this thoroughly.

### The four sources of environment configuration

Each source has a place. Inline for static known values. ConfigMap for non-sensitive shared config. Secret for credentials. Downward API for "tell me my own pod's name/IP/namespace." All four can coexist in one container.

| Source | YAML | Use case |
|---|---|---|
| Inline literals | `env: { name, value }` | Hardcoded values |
| ConfigMap | `envFrom: configMapRef` or `env: configMapKeyRef` | Non-sensitive config |
| Secret | `env: secretKeyRef` | Sensitive values |
| Downward API | `env: fieldRef` or `resourceFieldRef` | Pod's own metadata |

### The propagation rule that bites everyone

Recap: - Mounted as **environment variables** → running pods don't see new value until restart - Mounted as **files** → kubelet syncs ~60s, no restart needed

Fix when you've changed an env-var ConfigMap:

```
oc rollout restart deployment/<name>
```

### Hands-on Lab 3 — All four sources in one app

```
# 0. Confirm hello-s2i is alive
oc get deployment hello-s2i

# 1. Create a ConfigMap (non-sensitive)
oc create configmap hello-config \
  --from-literal=GREETING="Hello from vivek-c-dev" \
  --from-literal=LOG_LEVEL=info \
  --from-literal=FEATURE_FLAG=on

# 2. Create a Secret (sensitive)
oc create secret generic hello-secret \
  --from-literal=API_KEY=demo-token-abc123 \
  --from-literal=DB_PASSWORD='s3cr3t!'

# 3a. Bulk-import all ConfigMap keys
oc set env deployment/hello-s2i --from=configmap/hello-config

# 3b. Cherry-pick one Secret key (using oc patch for clarity)
oc patch deployment hello-s2i --type=json -p='[
  {"op": "add", "path": "/spec/template/spec/containers/0/env/-",
   "value": {"name": "API_KEY",
             "valueFrom": {"secretKeyRef":
               {"name":"hello-secret","key":"API_KEY"}}}}
]'

# 4. Add an inline literal
oc set env deployment/hello-s2i APP_VERSION="1.0.0"

# 5. Add Downward API — pod's own name and namespace
oc patch deployment hello-s2i --type=json -p='[
  {"op": "add", "path": "/spec/template/spec/containers/0/env/-",
   "value": {"name": "POD_NAME",
             "valueFrom": {"fieldRef": {"fieldPath": "metadata.name"}}}},
  {"op": "add", "path": "/spec/template/spec/containers/0/env/-",
   "value": {"name": "POD_NAMESPACE",
             "valueFrom": {"fieldRef": {"fieldPath": "metadata.namespace"}}}}
]'

# 6. Wait for rollout
oc rollout status deployment/hello-s2i

# 7. List every env var
oc set env deployment/hello-s2i --list

# 8. Prove it from inside the running container
POD=$(oc get pod -l deployment=hello-s2i -o name | head -1)
oc exec $POD -- env | grep -E \
  "GREETING|LOG_LEVEL|FEATURE_FLAG|API_KEY|APP_VERSION|POD_NAME|POD_NAMESPACE"
```

All seven values, sourced from four different mechanisms, **indistinguishable from each other to your application**. That uniformity is the whole point.

### Hands-on Lab 3 bonus — Watch the propagation rule fire

```
# Get current GREETING from the pod
oc exec $POD -- env | grep GREETING
# GREETING=Hello from vivek-c-dev

# Edit the ConfigMap
oc set data configmap/hello-config GREETING="Updated greeting"

# Same pod — UNCHANGED (the rule)
oc exec $POD -- env | grep GREETING
# GREETING=Hello from vivek-c-dev

# Restart to pick up the change
oc rollout restart deployment/hello-s2i
oc rollout status deployment/hello-s2i

# New pod — sees new value
POD=$(oc get pod -l deployment=hello-s2i -o name | head -1)
oc exec $POD -- env | grep GREETING
# GREETING=Updated greeting
```

### Mounting config as files — the alternative

When you want auto-refresh without restarts:

```
# Mount hello-config as files at /etc/config/
oc set volume deployment/hello-s2i \
  --add --name=config-vol \
  --type=configmap \
  --configmap-name=hello-config \
  --mount-path=/etc/config

# Wait for the rollout
oc rollout status deployment/hello-s2i

# Inside the pod
POD=$(oc get pod -l deployment=hello-s2i -o name | head -1)
oc exec $POD -- ls /etc/config
oc exec $POD -- cat /etc/config/GREETING

# Edit the ConfigMap again
oc set data configmap/hello-config GREETING="Auto-refreshed!"

# Wait ~60 seconds for kubelet to sync
sleep 70
oc exec $POD -- cat /etc/config/GREETING
# Auto-refreshed!         ← changed without a pod restart
```

Same pod, no restart, new value. The kubelet syncs the file every minute or so.

---

## 20. The complete Sandbox lab — end-to-end

Run this end-to-end and you'll exercise every concept in the guide on your actual cluster:

```
oc project vivek-c-dev

# 1. Deploy from Git via S2I
oc new-app nodejs:18-ubi9~https://github.com/sclorg/nodejs-ex.git --name=hello

# 2. Watch the build
oc logs -f bc/hello

# 3. Expose externally — IMPORTANT: use 'create route edge' not 'expose svc'
#    (oc expose creates a Route without TLS, browsers won't load it)
oc create route edge hello --service=hello --port=8080-tcp
oc get route hello

# 4. Add config
oc create configmap hello-config --from-literal=GREETING="Hi from vivek-c"
oc set env deployment/hello --from=configmap/hello-config

# 5. Add a secret
oc create secret generic hello-secret --from-literal=API_KEY=demo123
oc set env deployment/hello --from=secret/hello-secret

# 6. Trigger a manual rebuild
oc start-build hello

# 7. Watch the new deployment roll out
oc rollout status deployment/hello

# 8. Inspect everything
oc get all
oc get bc,is,configmap,secret
oc describe is hello
```

After this sequence you'll have touched: **Pods · Deployments · Services · Routes · BuildConfigs · ImageStreams · Builds · ConfigMaps · Secrets** — the full surface area of "deploying real apps on OpenShift."

---

# Appendix

## A. Sandbox capabilities and limits

The Developer Sandbox is a shared, multi-tenant OpenShift cluster managed by Red Hat. Limits at a glance:

### What works in the trial

- Pods, Deployments, Services
- ConfigMaps, Secrets
- Routes (with edge TLS)
- PVCs (small, default StorageClass, max ~5 GiB, RWO only)
- S2I builds from Git
- BuildConfigs, ImageStreams
- `oc` CLI and web console
- Topology view, logs, exec
- Pre-installed operators (catalog browse only)
- Helm charts (developer catalog)
- RBAC inside your project
- ServiceAccounts you create
- `oc port-forward`, `oc rsh`, debug pods
- Pipelines (Tekton) — limited runs

### What's blocked

- `oc new-project` (fixed projects only)
- cluster-admin operations
- Installing new operators
- Creating SCCs or ClusterRoles
- NodePort or LoadBalancer Service types
- Privileged or root-UID containers
- hostPath volumes, hostNetwork
- Custom StorageClasses
- Wildcard DNS / custom domains
- Custom CA certs
- Cluster monitoring access
- Modifying default network policy
- Large PVCs (>5 GiB)
- High replica counts (CPU/memory caps)

### Trial gotchas

- The cluster **idles your workloads** after inactivity. First request takes 10–30 seconds to wake them up.
- The **30-day clock** is rolling — log in once a month to keep your projects alive.
- No SSH to nodes, no host filesystem, no privileged containers.

To confirm exactly what your cluster lets you do:

```
oc auth can-i --list
oc describe quota
oc get storageclass
```

## B. Common gotchas

- **"Application is not available" on a Route** — usually means the Route was created without TLS (use `oc create route edge`, not `oc expose`). Browsers force HTTPS, the Route only handles HTTP, you get 503.
- **CrashLoopBackOff with "permission denied"** — the image expects to run as root. Use a non-root variant

(`unprivileged`, `nginxinc/nginx-unprivileged`, UBI-based images, `bitnami/*`).

- **Build runs but pod won't pull image** — check the ImageStream. Internal-registry pulls authenticate via the SA token automatically; external private pulls need a pull secret linked to the SA.
- **ConfigMap change not visible in pod** — env vars need a pod restart (`oc rollout restart`). Mounted files auto-refresh in ~60s.
- **PVC stuck in Pending** — most cloud block storage is RWO. If your Deployment has multiple replicas, only one pod can attach the volume. Use a StatefulSet with `volumeClaimTemplates`, or RWX storage like NFS.
- **Helm chart fails on Sandbox** — typically SCC blocking root images. Look for chart values that toggle non-root or use a different chart.

## C. Useful one-liners

```
# Who am I, where am I?
oc whoami
oc whoami --show-server
oc projects
oc project

# What can I do here?
oc auth can-i --list
oc auth can-i create deployments

# Find unhealthy pods across the project
oc get pods --field-selector=status.phase!=Running

# Tail logs from all pods of a deployment
oc logs -f deployment/myapp

# Get logs from the previous (crashed) container
oc logs deployment/myapp --previous

# One-screen project summary
oc status
oc get all

# Open a shell in a running container
oc rsh deployment/myapp

# Tunnel a Service to localhost
oc port-forward svc/myapp 8080:80

# Restart all pods of a Deployment
oc rollout restart deployment/myapp

# See what env vars a Deployment will give pods
oc set env deployment/myapp --list

# Add or remove env var
oc set env deployment/myapp DEBUG=true     # add
oc set env deployment/myapp DEBUG-          # remove (trailing dash)

# Apply a YAML and watch the result
oc apply -f manifest.yaml && oc get -f manifest.yaml -w

# Find every Deployment without a readiness probe
oc get deployment --all-namespaces -o json | \
  jq -r '.items[] | select(.spec.template.spec.containers[0].readinessProbe == null)
         | "\(.metadata.namespace)/\(.metadata.name)"'

# Force-delete a stuck pod
oc delete pod stuck-pod --grace-period=0 --force

# Show rollout history
oc rollout history deployment/myapp

# Roll back to previous version
oc rollout undo deployment/myapp
```

## End of Guide

This document covered the full path from container fundamentals to running production-style applications on OpenShift, calibrated for the Developer Sandbox trial cluster. Every command and YAML snippet here can be exercised on a free Sandbox account.

The next skills to learn beyond this guide are the **automation and operations** layer: - **Operators and OperatorHub** — installing and managing complex stateful services - **Tekton Pipelines** — cluster-native CI/CD - **ArgoCD / GitOps** — declarative cluster state from Git - **Network policies** — fine-grained pod-to-pod traffic control - **Service Mesh (Istio)** — mTLS, traffic management, observability across services

Each builds on the foundation you've now established. Master the basics here first, then everything else falls into place.